Раскрытие проблемы неподходящей локальности использования памяти

Майстренко Марат 323, СКИ ВМК МГУ

План доклада

01
Постановка проблемы

02
Неправильный код

03
Исправление кода

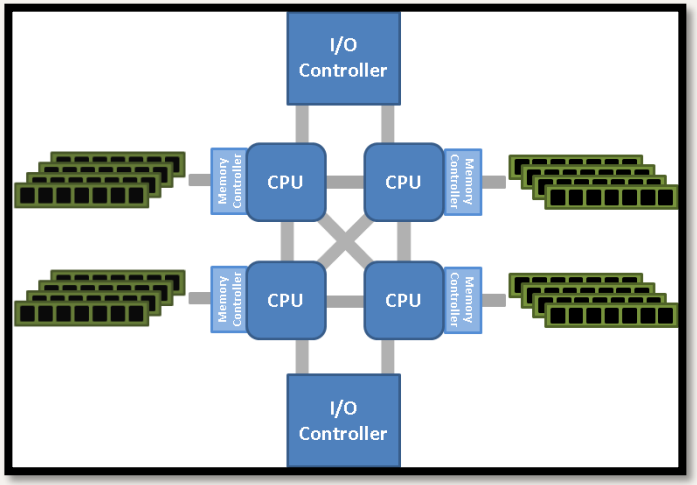
04
Измерения и анализ результатов

05
Профилирование с помощью **mpiP**

06
Заключение

Постановка проблемы

Обращение **в свою память – доли секунды**, но обращение **в память другого вычислительного узла** – довольно **затратная задача**, порождающая новые накладные расходы и создающая узкое место в программе.



Неправильный код

Задача – массив делится между P процессами, каждый процесс подсчитывает свою часть, а затем формируется общая сумма

Основная проблема — **централизованный доступ всех процессов к общим данным** во время вычислений.

```
void bad_access(int N, int rank, int size, MPI_Win win, int* shared_array) {
    int local_sum = 0;

    MPI_Win_fence(0, win);

    for (int i = rank; i < N; i += size) {
        int value;
        MPI_Get(&value, 1, MPI_INT, 0, i, 1, MPI_INT, win);
        local_sum += value;
    }

    MPI_Win_fence(0, win);
    MPI_Accumulate(&local_sum, 1, MPI_INT, 0, N, 1, MPI_INT, MPI_SUM, win);
    MPI_Win_fence(0, win);
}
```

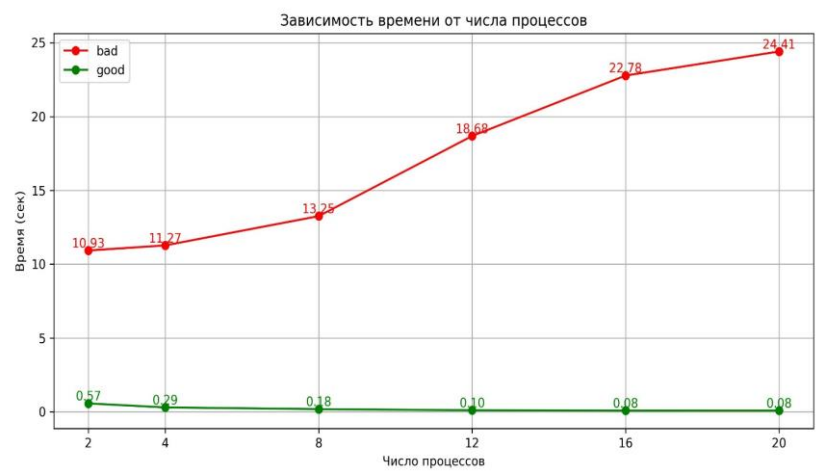
Исправление кода

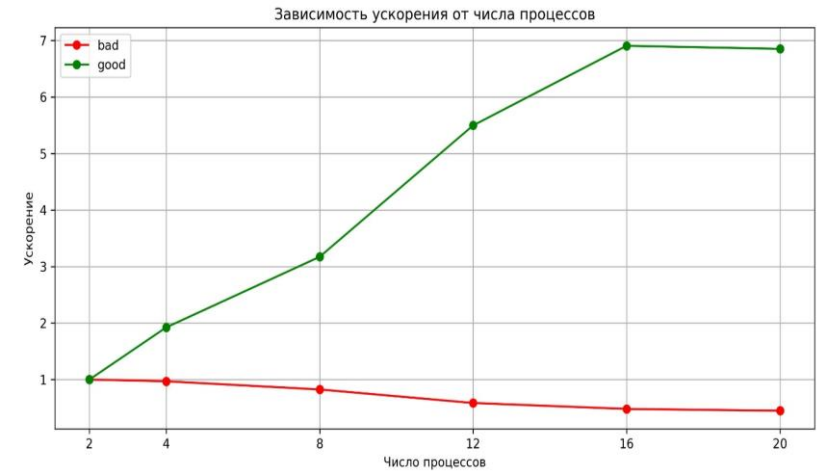
<u>Решение</u> – заранее распределяем данные между процессами

Устраняем проблему **централизованного доступа**

```
void good_access(int N, int rank, int size, MPI_Win win, int* shared_array) {
    int block_size = N / size;
    int start = rank * block_size;
    int end = (rank == size - 1) ? N : start + block_size;
    int my_count = end - start;
    std::vector<int> local_block(my_count);

    MPI_Win_fence(0, win);
    MPI_Get(local_block.data(), my_count, MPI_INT, 0, start, my_count, MPI_INT, win);
    MPI_Win_fence(0, win);

    int local_sum = 0;
    for (int v : local_block)
        local_sum += v;

    MPI_Win_fence(0, win);
    MPI_Accumulate(&local_sum, 1, MPI_INT, 0, N, 1, MPI_INT, MPI_SUM, win);
    MPI_Win_fence(0, win);
}
```

Измерения и анализ результатов





good_access (оптимизированная версия):

- При 20 процессах достигается **максимальное ускорение** (~0.08 сек), что почти **300-кратное** ускорение по сравнению с плохой версией

bad_access (неоптимизированная версия):

- Масштабируемость отсутствует: с ростом числа процессов время **увеличивается**

Профилирование с помощью mpiP

Программа имеет **высокую зависимость от MPI-коммуникаций**:

среднее время MPI – *98.02%* общего времени выполнения (487 из 496 секунд).

```
---------------------------------------------------------------------------
@--- Aggregate Time (top twenty, descending, milliseconds) ----------------
---------------------------------------------------------------------------
Call                 Site       Time    App%    MPI%      Count    COV
Win_fence               1   2.51e+05   50.63   51.65         20   0.92
Get                    14   2.27e+05   45.75   46.68   50000000   1.02
Win_create              3   7.52e+03    1.51    1.55         20   0.23
Win_fence              10        400    0.08    0.08         20   0.34
Win_fence              15        127    0.03    0.03         20   0.34
Get                     9       42.1    0.01    0.01         20   1.03
Win_free               16       21.5    0.00    0.00         20   0.20
Win_fence               8       2.23    0.00    0.00         20   0.49
Accumulate              6       1.53    0.00    0.00         20   0.42
Win_fence               5       1.48    0.00    0.00         20   0.59
Win_fence              12       1.42    0.00    0.00         20   0.22
Barrier                 4      0.778    0.00    0.00         20   0.33
Barrier                17      0.607    0.00    0.00         20   0.22
Win_fence              11      0.315    0.00    0.00         20   0.14
Accumulate             13       0.26    0.00    0.00         20   1.23
Barrier                 7      0.201    0.00    0.00         20   0.25
Barrier                 2      0.148    0.00    0.00         20   0.37
```

Заключение

Плохая локальность данных приводит к **лишним накладным расходам**. Грамотно локализованный и **упорядоченный обмен** позволяет **их избежать**.

Это подчёркивает **важность тщательного проектирования** стратегии доступа и обмена данными, минимизируя дорогие синхронизации и использования затратных средств MPI.